# Блочное LU-разложение

Курс «Параллельные алгоритмы» · Алгоритм 37 пособия Головашкина

$$A = L\,U$$

Теория · Последовательный алгоритм · Векторизация · OpenMP · MPI

Intel Core i5-1135G7 · MinGW GCC 9.2 · MS-MPI

# Содержание


## Теория LU-разложения

1. Треугольные матрицы
2. Определение LU-разложения
3. Метод Гаусса в матричной форме
4. Поэлементные формулы (схема Дулиттла)
5. Существование и единственность
6. Блочная форма и дополнение Шура
7. Пример: разложение матрицы 3 × 3
8. Трудоёмкость
9. Устойчивость и выбор ведущего элемента
10. Сводка формул

Литература

## Этап 1. Последовательное блочное LU-разложение

1. Постановка задачи
2. Вывод алгоритма
3. Шаги алгоритма в поэлементной записи
4. Вычислительная сложность
5. Решение системы и проверка
6. Тестовая матрица и измерение погрешности
7. Соответствие коду

Литература

## Этап 2. Векторизация блочного LU-разложения

1. Что такое векторизация
2. Все шаги алгоритма — одна операция saxpy
3. Сборки и флаги компилятора
4. Модель ожидаемого ускорения
5. Результаты
6. Отличия от этапа 1

## Этап 3. OpenMP: блочное LU-разложение

1. Модель OpenMP
2. Какие циклы можно распараллелить
3. Модель ускорения
4. Результаты

## Этап 4. MPI-версия блочного LU-разложения

1. Модель MPI
2. Распределение данных

3. Параллельный алгоритм одной итерации
4. Анализ стоимости
5. Замер времени
6. Результаты

# Теория LU-разложения

Этот документ содержит математическую основу всех четырёх программ: что такое LU-разложение, как оно получается из метода Гаусса, полный вывод формул, условия существования, блочная форма, трудоёмкость и устойчивость. Реализация алгоритма описана в этапе 1.

Обозначения: $A = (a_{ij}) \in \mathbb{R}^{n\times n}$; индексы в этом документе нумеруются **с единицы**, как принято в математике (в коде — с нуля); $e_k$ — $k$-й столбец единичной матрицы $I$; $A_k = A_{1:k,\,1:k}$ — ведущая главная подматрица порядка $k$.

## 1. Треугольные матрицы

Матрица $L$ **нижняя треугольная**, если $l_{ij} = 0$ при $i < j$, и **верхняя треугольная** ($U$), если $u_{ij} = 0$ при $i > j$. Нижняя треугольная матрица с единицами на диагонали называется **унитреугольной** (unit lower triangular).

Для дальнейшего понадобятся четыре свойства.

**Свойство 1.** *Произведение нижних треугольных матриц — нижняя треугольная матрица.*

Пусть $L, M$ — нижние треугольные. Элемент произведения

$$(LM)_{ij} = \sum_{s=1}^{n} l_{is}\, m_{sj}.$$

Слагаемое ненулевое только при $s \le i$ (иначе $l_{is} = 0$) и $s \ge j$ (иначе $m_{sj} = 0$). При $i < j$ таких $s$ нет, значит $(LM)_{ij} = 0$. Диагональ: $(LM)_{ii} = l_{ii}\, m_{ii}$, поэтому произведение унитреугольных матриц — унитреугольная матрица.

**Свойство 2.** *Определитель треугольной матрицы равен произведению диагональных элементов:*

$$\det L = \prod_{i=1}^{n} l_{ii}.$$

(Разложение по первой строке для нижней треугольной матрицы оставляет одно слагаемое $l_{11}\det L_{2:n,\,2:n}$, далее по индукции.) В частности, унитреугольная матрица всегда невырожденна: $\det L = 1$.

**Свойство 3.** *Обратная к невырожденной нижней треугольной матрице — нижняя треугольная; к унитреугольной — унитреугольная.*

Столбец $x = (L^{-1})_{:,j}$ — решение системы $Lx = e_j$. Прямая подстановка даёт $x_1 = \dots = x_{j-1} = 0$ (правая часть в этих строках нулевая) и $x_j = 1/l_{jj}$. Значит, выше диагонали у $L^{-1}$ нули, а на диагонали — $1/l_{jj}$.

**Свойство 4.** *Треугольные системы решаются за $O(n^2)$ операций.* Система $Ly = b$ решается прямой подстановкой, $Ux = y$ — обратной:

$$y_i = \frac{1}{l_{ii}}\Bigl(b_i - \sum_{j=1}^{i-1} l_{ij}\, y_j\Bigr), \quad i = 1,\dots,n; \qquad x_i = \frac{1}{u_{ii}}\Bigl(y_i - \sum_{j=i+1}^{n} u_{ij}\, x_j\Bigr), \quad i = n,\dots,1.$$

Строка $i$ требует $i-1$ умножений и столько же вычитаний, всего $\sum_{i=1}^{n} 2(i-1) + n = n^2$ флопов.

## 2. Определение LU-разложения

**LU-разложением** матрицы $A$ называется представление

$$A = LU,$$

где $L$ — нижняя **унитреугольная**, $U$ — верхняя треугольная матрица. Требование $l_{ii} = 1$ (вариант Дулиттла, Doolittle) делает разложение единственным; в варианте Краута (Crout) единицы ставят на диагональ $U$.

Зачем оно нужно. Если $A = LU$, то система $Ax = b$ заменяется двумя треугольными:

$$Ax = b \iff L\,(Ux) = b \iff \begin{cases} L\,y = b, \\ U\,x = y. \end{cases}$$

Разложение стоит $\frac{2}{3}n^3$ флопов (см. раздел 8), а каждая треугольная система — $n^2$. Поэтому для $m$ правых частей с одной матрицей метод выгоден: $\frac{2}{3}n^3 + 2mn^2$ вместо $m \cdot \frac{2}{3}n^3$ при повторном методе Гаусса.

## 3. Метод Гаусса в матричной форме

### Один шаг исключения

Метод Гаусса последовательно обнуляет элементы под диагональю. Обозначим $A^{(1)} = A$. Перед шагом $k$ матрица $A^{(k)}$ уже имеет нули под диагональю в столбцах $1, \dots, k-1$:

$$A^{(k)} = \begin{pmatrix} a_{11}^{(1)} & \cdots & a_{1,k}^{(1)} & \cdots & a_{1n}^{(1)} \\ & \ddots & \vdots & & \vdots \\ & & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ & & \vdots & & \vdots \\ & & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} \end{pmatrix}.$$

Элемент $a_{kk}^{(k)}$ называется **ведущим** (pivot). Если $a_{kk}^{(k)} \neq 0$, вычислим **множители**

$$l_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \qquad i = k+1, \dots, n,$$

и вычтем из каждой строки $i > k$ строку $k$, умноженную на $l_{ik}$:

$$\boxed{a_{ij}^{(k+1)} = a_{ij}^{(k)} - l_{ik}\, a_{kj}^{(k)}, \qquad i, j = k+1, \dots, n.}$$

Строки $1, \dots, k$ не меняются, а элементы $a_{ik}^{(k+1)}$, $i > k$, становятся нулями (именно для этого выбран множитель: $a_{ik}^{(k)} - l_{ik} a_{kk}^{(k)} = 0$). После $n-1$ шагов получается верхняя треугольная матрица $U = A^{(n)}$.

### Матрица Гаусса

Шаг $k$ — это умножение слева на **матрицу Гаусса** (элементарную нижнюю треугольную матрицу)

$$M_k = I - \ell_k\, e_k^T, \qquad \ell_k = (0, \dots, 0,\ l_{k+1,k},\ \dots,\ l_{nk})^T,$$

у которой в векторе $\ell_k$ первые $k$ компонент — нули:

$$M_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & 1 \end{pmatrix}.$$

Действительно, $(M_k A^{(k)})_{i,:} = A_{i,:}^{(k)} - l_{ik}\, A_{k,:}^{(k)}$ — это и есть вычитание строк. Итак,

$$A^{(k+1)} = M_k\, A^{(k)}, \qquad U = A^{(n)} = M_{n-1} M_{n-2} \cdots M_1\, A.$$

### Обратная к матрице Гаусса

**Лемма 1.** $M_k^{-1} = I + \ell_k\, e_k^T$.

*Доказательство.* Перемножим:

$$(I - \ell_k e_k^T)(I + \ell_k e_k^T) = I - \ell_k e_k^T + \ell_k e_k^T - \ell_k\,(e_k^T \ell_k)\, e_k^T = I,$$

так как $e_k^T \ell_k = (\ell_k)_k = 0$ (у $\ell_k$ нули в первых $k$ позициях). ■

Обращение матрицы Гаусса сводится к смене знака множителей.

### Сборка матрицы L

Из $U = M_{n-1}\cdots M_1 A$ получаем

$$A = M_1^{-1} M_2^{-1} \cdots M_{n-1}^{-1}\, U = L\,U, \qquad L = M_1^{-1} M_2^{-1} \cdots M_{n-1}^{-1}.$$

**Лемма 2.** $L = I + \sum_{k=1}^{n-1} \ell_k\, e_k^T$, то есть

$$L = \begin{pmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & l_{n3} & \cdots & 1 \end{pmatrix}.$$

*Доказательство.* Для $i < j$:

$$(I + \ell_i e_i^T)(I + \ell_j e_j^T) = I + \ell_i e_i^T + \ell_j e_j^T + \ell_i\,(e_i^T \ell_j)\, e_j^T,$$

а $e_i^T \ell_j = (\ell_j)_i = 0$, потому что $i < j$ и у $\ell_j$ нули в позициях $1, \dots, j$. По индукции произведение $M_1^{-1}\cdots M_{n-1}^{-1}$ равно $I + \sum_k \ell_k e_k^T$. ■

Вывод: $L$ **состоит из множителей метода Гаусса**, записанных под диагональю, а $U$ — это матрица, полученная в конце прямого хода. Никаких дополнительных вычислений для $L$ не нужно: множитель $l_{ik}$ можно хранить на месте обнулённого элемента $a_{ik}$. Именно так программа хранит $L$ и $U$ в одном массиве.

> Порядок важен: $L = M_1^{-1}\cdots M_{n-1}^{-1}$ получается простым «складыванием» множителей, а $M_{n-1}\cdots M_1 = L^{-1}$ — нет: в этом произведении появляются перекрёстные слагаемые $\ell_j (e_j^T \ell_i) e_i^T \neq 0$ при $j > i$.

## 4. Поэлементные формулы (схема Дулиттла)

Те же формулы можно получить напрямую, без матриц Гаусса, приравняв элементы обеих частей $A = LU$. Так как $l_{is} = 0$ при $s > i$ и $u_{sj} = 0$ при $s > j$,

$$a_{ij} = \sum_{s=1}^{n} l_{is}\, u_{sj} = \sum_{s=1}^{\min(i,j)} l_{is}\, u_{sj}.$$

**Случай** $i \le j$ (элемент на диагонали или выше). Тогда $\min(i,j) = i$ и последнее слагаемое — $l_{ii}u_{ij} = u_{ij}$:

$$a_{ij} = \sum_{s=1}^{i-1} l_{is}\, u_{sj} + u_{ij} \quad\Longrightarrow\quad \boxed{u_{ij} = a_{ij} - \sum_{s=1}^{i-1} l_{is}\, u_{sj}, \qquad j = i, \dots, n.}$$

**Случай** $i > j$ (элемент ниже диагонали). Тогда $\min(i,j) = j$, последнее слагаемое — $l_{ij}u_{jj}$:

$$a_{ij} = \sum_{s=1}^{j-1} l_{is}\,u_{sj} + l_{ij}\,u_{jj} \quad\Longrightarrow\quad \boxed{\,l_{ij} = \frac{1}{u_{jj}}\Bigl(a_{ij} - \sum_{s=1}^{j-1} l_{is}\,u_{sj}\Bigr), \qquad i = j+1,\dots,n.\,}$$

### Порядок вычислений

Правые части содержат только элементы $L$ и $U$ с меньшими индексами суммирования, поэтому их можно вычислить последовательно. Классический порядок — чередование строк $U$ и столбцов $L$:

$$\underbrace{u_{11},\dots,u_{1n}}_{\text{строка 1 } U},\ \underbrace{l_{21},\dots,l_{n1}}_{\text{столбец 1 } L},\ \underbrace{u_{22},\dots,u_{2n}}_{\text{строка 2 } U},\ \underbrace{l_{32},\dots,l_{n2}}_{\text{столбец 2 } L},\ \dots$$

Но возможны и другие порядки — например, **по столбцам** (сначала весь столбец $j$ матрицы $U$, затем столбец $j$ матрицы $L$): это jki-форма, или LU-разложение через gaxpy. В программах применяется блочный алгоритм с модификацией матрицы внешним произведением (см. раздел 6 и этап 1).

### Связь с методом Гаусса

Промежуточные элементы метода Гаусса — это частичные суммы тех же формул:

$$a_{ij}^{(k)} = a_{ij} - \sum_{s=1}^{k-1} l_{is}\,u_{sj}.$$

(Доказывается индукцией по $k$ из формулы пересчёта $a_{ij}^{(k+1)} = a_{ij}^{(k)} - l_{ik}a_{kj}^{(k)}$ и равенства $a_{kj}^{(k)} = u_{kj}$.) При $k = i$ для $i \le j$ это $u_{ij}$, а при $k = j$ для $i > j$ — числитель формулы для $l_{ij}$. Значит, **схема Дулиттла и метод Гаусса выполняют одни и те же арифметические операции**, различается только порядок: в методе Гаусса слагаемые вычитаются «сразу во всю оставшуюся матрицу» (right-looking), в схеме Дулиттла — «накапливаются» при вычислении очередного элемента (left- или up-looking). Поэтому и все шесть форм циклов $ijk$ дают один и тот же результат в точной арифметике.

## 5. Существование и единственность

**Теорема.** *Пусть все ведущие главные миноры матрицы $A$ порядков $1,\dots,n-1$ отличны от нуля:*

$$\det A_k \neq 0, \qquad k = 1,\dots,n-1.$$

*Тогда LU-разложение с унитреугольной $L$ существует и единственно.*

*Доказательство* (индукция по $n$). При $n = 1$: $L = (1)$, $U = (a_{11})$. Пусть утверждение верно для порядка $n-1$. Запишем $A$ в окаймлённом виде:

$$A = \begin{pmatrix} A_{n-1} & b \\ c^T & a_{nn} \end{pmatrix}, \qquad L = \begin{pmatrix} L_{n-1} & 0 \\ x^T & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} U_{n-1} & y \\ 0 & u_{nn} \end{pmatrix}.$$

Перемножив блоки, получаем, что $A = LU$ равносильно четырём равенствам:

$$\begin{aligned} A_{n-1} &= L_{n-1}\,U_{n-1}, \\ b &= L_{n-1}\,y, \\ c^T &= x^T\,U_{n-1}, \\ a_{nn} &= x^T y + u_{nn}. \end{aligned}$$

1. Ведущие миноры $A_{n-1}$ порядков $1,\dots,n-2$ — это миноры $A$, они ненулевые; по предположению индукции $A_{n-1} = L_{n-1}U_{n-1}$ существует и единственно.
2. $L_{n-1}$ унитреугольная, значит невырожденная: $y = L_{n-1}^{-1}b$ определяется однозначно.
3. $\det U_{n-1} = \det L_{n-1}\det U_{n-1} = \det A_{n-1} \neq 0$, значит $U_{n-1}$ невырожденна: $x^T = c^T U_{n-1}^{-1}$ определяется однозначно.

4. $u_{nn} = a_{nn} - x^T y$ определяется однозначно. ■

Заметим, что $\det A_n$ (то есть $\det A$) в условии не участвует: вырожденная матрица тоже может иметь LU-разложение, но тогда $u_{nn} = 0$.

### Ведущие элементы через миноры

Из $A_k = L_k U_k$ (ведущие подматрицы перемножаются независимо от остальных блоков) и свойства 2:

$$\det A_k = \det L_k \cdot \det U_k = \prod_{i=1}^{k} u_{ii} \quad \Longrightarrow \quad \boxed{u_{kk} = \frac{\det A_k}{\det A_{k-1}}, \qquad \det A_0 := 1.}$$

Следствия:

- метод Гаусса без перестановок не встречает нулевого ведущего элемента **тогда и только тогда**, когда все ведущие миноры $\det A_1, \ldots, \det A_{n-1}$ ненулевые;
- определитель вычисляется как побочный результат разложения:

$$\det A = \prod_{k=1}^{n} u_{kk}.$$

**Пример матрицы без LU-разложения:** $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ невырожденна, но $\det A_1 = 0$. Из $a_{11} = l_{11} u_{11} = u_{11} = 0$ и $a_{21} = l_{21} u_{11} = 1$ получаем противоречие $0 = 1$. После перестановки строк разложение существует — см. раздел 9.

## 6. Блочная форма и дополнение Шура

### Один шаг как разложение на блоки

Выделим первую строку и первый столбец:

$$A = \begin{pmatrix} a_{11} & w^T \\ v & B \end{pmatrix}, \qquad v, w \in \mathbb{R}^{n-1},\ B \in \mathbb{R}^{(n-1)\times(n-1)}.$$

При $a_{11} \neq 0$ непосредственно проверяется равенство

$$\begin{pmatrix} a_{11} & w^T \\ v & B \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ v/a_{11} & I \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & S \end{pmatrix}, \qquad \boxed{S = B - \frac{v\, w^T}{a_{11}}.}$$

Матрица $S$ называется **дополнением Шура** элемента $a_{11}$. Это ровно матрица $A^{(2)}_{2:n,\,2:n}$ после первого шага метода Гаусса: $s_{ij} = b_{ij} - l_{i1} w_j$. Если $S = L_S U_S$, то

$$A = \begin{pmatrix} 1 & 0 \\ v/a_{11} & L_S \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & U_S \end{pmatrix},$$

то есть LU-разложение — это **рекурсия**: первая строка $U$ — первая строка $A$, первый столбец $L$ — $v/a_{11}$, остальное — разложение дополнения Шура. Обновление $B - vw^T/a_{11}$ — **внешнее произведение** (rank-1 update), это right-looking-форма алгоритма.

### Блоки размера r

Тот же приём с блоком $A_{11}$ порядка $r$:

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ 0 & U_{22} \end{pmatrix}.$$

Перемножив блоки, получаем четыре уравнения и порядок их решения:

$$
\begin{aligned}
A_{11} &= L_{11}U_{11} && \Rightarrow \text{ неблочное LU блока } r \times r,\\
A_{12} &= L_{11}U_{12} && \Rightarrow\ U_{12} = L_{11}^{-1}A_{12} \quad \text{(треугольная система)},\\
A_{21} &= L_{21}U_{11} && \Rightarrow\ L_{21} = A_{21}U_{11}^{-1} \quad \text{(треугольная система)},\\
A_{22} &= L_{21}U_{12} + L_{22}U_{22} && \Rightarrow\ L_{22}U_{22} = \underbrace{A_{22} - L_{21}U_{12}}_{\text{дополнение Шура}}.
\end{aligned}
$$

Последняя строка — рекурсия для матрицы порядка $n - r$. Основная работа — матричное умножение $L_{21}U_{12}$, которое хорошо использует кэш и векторные инструкции. В этом и состоит смысл блочных алгоритмов.

### Right-looking и left-looking

Блочная схема выше — **right-looking**: после разложения блочного столбца дополнение Шура сразу пересчитывается для **всей** правой нижней подматрицы. Именно она реализована в программах (Алгоритм 37 из пособия Головашкина, параграф 2.4). Существует и **left-looking**-вариант (jki): блочный столбец $K$ не трогается до своей очереди, а затем одним действием получает все накопленные обновления от столбцов левее:

$$
A_{k:n,\,K} - L_{k:n,\,1:k}\,U_{1:k,\,K} = A_{k:n,\,K} - \sum_{\text{блоки } J<K} L_{k:n,\,J}\,U_{J,\,K}.
$$

Сумма справа — это ровно все обновления right-looking-схемы, которые пришлись бы на столбцы $K$, только применённые одним блочным GAXPY. Арифметика та же, отличается порядок обращений к памяти: left-looking пишет только в текущий блочный столбец и читает всё остальное, right-looking пишет во всю оставшуюся подматрицу.

## 7. Пример: разложение матрицы 3 × 3

Разложим

$$
A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}.
$$

### Методом Гаусса

**Шаг 1.** Ведущий элемент $a_{11} = 2$. Множители: $l_{21} = 4/2 = 2$, $l_{31} = 8/2 = 4$. Из строки 2 вычитаем удвоенную строку 1, из строки 3 — учетверённую:

$$
A^{(2)} = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 3 - 2\cdot 1 & 3 - 2\cdot 1 \\ 0 & 7 - 4\cdot 1 & 9 - 4\cdot 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 3 & 5 \end{pmatrix}.
$$

**Шаг 2.** Ведущий элемент $a_{22}^{(2)} = 1$. Множитель $l_{32} = 3/1 = 3$:

$$
A^{(3)} = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 5 - 3\cdot 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix} = U.
$$

Множители образуют $L$:

$$
L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix}.
$$

### По формулам Дулиттла

$$
\begin{aligned}
&\text{строка } 1\ U: && u_{11}=2, \quad u_{12}=1, \quad u_{13}=1;\\
&\text{столбец } 1\ L: && l_{21}=\tfrac{4}{2}=2, \quad l_{31}=\tfrac{8}{2}=4;\\
&\text{строка } 2\ U: && u_{22}=3-l_{21}u_{12}=3-2=1, \quad u_{23}=3-l_{21}u_{13}=1;\\
&\text{столбец } 2\ L: && l_{32}=\tfrac{1}{u_{22}}\,(7-l_{31}u_{12})=7-4=3;\\
&\text{строка } 3\ U: && u_{33}=9-l_{31}u_{13}-l_{32}u_{23}=9-4-3=2.
\end{aligned}
$$

Результат тот же.

### Проверка

$$
LU=\begin{pmatrix} 1\cdot 2 & 1\cdot 1 & 1\cdot 1 \\ 2\cdot 2 & 2\cdot 1+1 & 2\cdot 1+1 \\ 4\cdot 2 & 4\cdot 1+3\cdot 1 & 4\cdot 1+3\cdot 1+2 \end{pmatrix}=\begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}=A.\ \checkmark
$$

Ведущие элементы через миноры: $\det A_1=2$, $\det A_2=2\cdot 3-1\cdot 4=2$, $\det A=2\cdot 1\cdot 2=4$, откуда $u_{11}=2$, $u_{22}=2/2=1$, $u_{33}=4/2=2$. $\checkmark$

### Решение системы

Возьмём $b=A\,(1,1,1)^T=(4,10,24)^T$ — так же строится правая часть в программах.

Прямой ход, $Ly=b$:

$$
y_1=4, \qquad y_2=10-2\cdot 4=2, \qquad y_3=24-4\cdot 4-3\cdot 2=2.
$$

Обратный ход, $Ux=y$:

$$
x_3=\frac{2}{2}=1, \qquad x_2=\frac{2-1\cdot 1}{1}=1, \qquad x_1=\frac{4-1\cdot 1-1\cdot 1}{2}=1.
$$

Получено точное решение $x=(1,1,1)^T$.

## 8. Трудоёмкость

Считаем отдельно умножения, сложения/вычитания и деления — каждое как один флоп.

На шаге $k$ метода Гаусса ($k=1,\dots,n-1$) обрабатывается подматрица порядка $m=n-k$:

- $m$ делений — множители $l_{ik}$;
- $m^2$ умножений и $m^2$ вычитаний — пересчёт $a_{ij}^{(k+1)}$.

Итого

$$
\sum_{k=1}^{n-1}\bigl(m+2m^2\bigr)\Big|_{m=n-k}=\sum_{m=1}^{n-1}m+2\sum_{m=1}^{n-1}m^2=\frac{n(n-1)}{2}+2\cdot\frac{(n-1)\,n\,(2n-1)}{6}.
$$

Упростим:

$$
\frac{n(n-1)}{2}+\frac{n(n-1)(2n-1)}{3}=\frac{n(n-1)\bigl(3+2(2n-1)\bigr)}{6}=\frac{n(n-1)(4n+1)}{6}=\boxed{\frac{2}{3}n^3-\frac{1}{2}n^2-\frac{1}{6}n.}
$$

Главный член — $\frac{2}{3}n^3$. Все формы алгоритма (Гаусс, Дулиттл, шесть порядков циклов, блочные варианты) выполняют одни и те же операции, поэтому имеют одну и ту же трудоёмкость.

| Задача | Флопы |
|---|---|
| LU-разложение | $\frac{2}{3}n^3 + O(n^2)$ |
| решение $Ly = b$ и $Ux = y$ | $2n^2$ |
| решение $Ax = b$ с $m$ правыми частями | $\frac{2}{3}n^3 + 2mn^2$ |
| определитель $\det A = \prod u_{kk}$ | $\frac{2}{3}n^3 + n$ |
| обратная матрица ($n$ правых частей $e_j$) | $\frac{2}{3}n^3 + 2n^3 = \frac{8}{3}n^3$ (с учётом нулей в $e_j$ — $2n^3$) |

Память: $L$ и $U$ помещаются на место $A$ — $n^2$ чисел, дополнительная память не нужна.

## 9. Устойчивость и выбор ведущего элемента

### Малый ведущий элемент

Даже если ведущий элемент не равен нулю, он может быть **мал**, и тогда метод неустойчив. Классический пример ($\varepsilon \ll 1$):

$$A = \begin{pmatrix} \varepsilon & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 1/\varepsilon & 1 \end{pmatrix} \begin{pmatrix} \varepsilon & 1 \\ 0 & 1 - 1/\varepsilon \end{pmatrix}.$$

При $\varepsilon = 10^{-20}$ в арифметике double $1 - 10^{20}$ округляется до $-10^{20}$, и вычисленное произведение $\hat{L}\hat{U} = \begin{pmatrix} \varepsilon & 1 \\ 1 & 0 \end{pmatrix}$ — элемент $a_{22} = 1$ полностью потерян. Причина — огромный множитель $l_{21} = 1/\varepsilon$: элементы $L$ и $U$ во много раз больше элементов $A$.

### Частичный выбор ведущего элемента

На шаге $k$ строку с наибольшим по модулю элементом столбца $k$ переставляют на место ведущей. Перестановки накапливаются в матрице перестановок $P$:

$$PA = LU, \qquad |l_{ij}| \le 1.$$

Такое разложение существует для **любой** невырожденной матрицы. Устойчивость характеризует **коэффициент роста**

$$\rho = \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|}.$$

Обратная ошибка метода Гаусса пропорциональна $\rho\, n \varepsilon$ (Higham, гл. 9): при малом $\rho$ метод устойчив. С частичным выбором $\rho \le 2^{n-1}$, но на практике $\rho$ почти всегда мал.

### Когда выбор ведущего элемента не нужен

**Теорема.** *Если $A$ имеет строгое диагональное преобладание по строкам,*

$$|a_{ii}| > \sum_{j \ne i} |a_{ij}|, \qquad i = 1, \dots, n,$$

*то метод Гаусса без перестановок выполним (все ведущие элементы ненулевые).*

*Доказательство.* Ведущий элемент $a_{11} \ne 0$, так как $|a_{11}| > 0$. Покажем, что дополнение Шура $S = B - vw^T/a_{11}$ (обозначения раздела 6) снова имеет строгое диагональное преобладание; дальше — индукция.

Для строки $i$ матрицы $B$ (строка $i+1$ матрицы $A$) условие преобладания: $|b_{ii}| > |v_i| + \sum_{j \ne i} |b_{ij}|$. Для первой строки $A$: $\sum_j |w_j| < |a_{11}|$. Оценим строку $i$ матрицы $S$:

$$|s_{ii}| \ge |b_{ii}| - \frac{|v_i|\,|w_i|}{|a_{11}|}, \qquad \sum_{j\ne i}|s_{ij}| \le \sum_{j\ne i}|b_{ij}| + \frac{|v_i|}{|a_{11}|}\sum_{j\ne i}|w_j|.$$

Вычитая,

$$|s_{ii}| - \sum_{j\ne i}|s_{ij}| \;\ge\; \underbrace{|b_{ii}| - \sum_{j\ne i}|b_{ij}|}_{>\,|v_i|} \;-\; |v_i|\,\underbrace{\frac{\sum_j |w_j|}{|a_{11}|}}_{<\,1} \;>\; |v_i| - |v_i| = 0. \qquad \blacksquare$$

Аналогичное утверждение верно для преобладания по столбцам (тогда ещё и $|l_{ij}| \le 1$) и для симметричных положительно определённых матриц (там вместо LU обычно используют разложение Холецкого $A = LL^T$).

**Применение к программам.** Тестовая матрица строится как $a_{ij} \in [-\frac{1}{2}, \frac{1}{2}]$ плюс $\frac{n}{4}$ на диагонали. Диагональ примерно равна **средней** сумме модулей внедиагональных элементов строки ($\frac{n-1}{4}$), поэтому строгое преобладание выполняется не для всех строк и теорема формально не применима. Тем не менее матрица далека от вырожденной, ведущие элементы остаются порядка $\frac{n}{4}$, и разложение без перестановок даёт ошибки порядка $10^{-14}$ (см. этап 1, раздел 7).

### Оценка обратной ошибки

Для вычисленных в арифметике с плавающей точкой множителей $\hat L, \hat U$ справедливо (Higham, теорема 9.3)

$$\hat L\hat U = A + \Delta A, \qquad |\Delta A| \le \gamma_n\,|\hat L|\,|\hat U|, \qquad \gamma_n = \frac{n\varepsilon}{1 - n\varepsilon},$$

где модуль и неравенство понимаются поэлементно, $\varepsilon \approx 1{,}1 \cdot 10^{-16}$ — машинная точность double. Если элементы $\hat L$ и $\hat U$ не намного больше элементов $A$ (малый рост $\rho$), разложение **обратно устойчиво**: вычисленные множители — точное разложение близкой матрицы. Для решения системы отсюда следует

$$\frac{\|\hat x - x\|}{\|x\|} \lesssim \kappa(A)\cdot c\,n\,\rho\,\varepsilon, \qquad \kappa(A) = \|A\|\,\|A^{-1}\|,$$

то есть погрешность растёт с размерностью $n$ и числом обусловленности. Именно этот рост измеряется на этапе 1.

## 10. Сводка формул

| Что | Формула |
|---|---|
| разложение | $A = LU,\ l_{ii} = 1$ |
| шаг Гаусса | $l_{ik} = a_{ik}^{(k)}/a_{kk}^{(k)},\ a_{ij}^{(k+1)} = a_{ij}^{(k)} - l_{ik}a_{kj}^{(k)}$ |
| матрица Гаусса | $M_k = I - \ell_k e_k^T,\ M_k^{-1} = I + \ell_k e_k^T$ |
| матрица $L$ | $L = I + \sum_k \ell_k e_k^T$ |
| элементы $U$ | $u_{ij} = a_{ij} - \sum_{s<i} l_{is}u_{sj},\ i \le j$ |
| элементы $L$ | $l_{ij} = \big(a_{ij} - \sum_{s<j} l_{is}u_{sj}\big)/u_{jj},\ i > j$ |
| существование | $\det A_k \ne 0, k = 1,\dots,n-1$ |
| ведущие элементы | $u_{kk} = \det A_k / \det A_{k-1}$ |
| определитель | $\det A = \prod_k u_{kk}$ |
| дополнение Шура | $S = A_{22} - A_{21}A_{11}^{-1}A_{12} = A_{22} - L_{21}U_{12}$ |
| трудоёмкость | $\frac{2}{3}n^3 - \frac{1}{2}n^2 - \frac{1}{6}n$ |
| обратная ошибка | $\hat L\hat U = A + \Delta A,\ \|\Delta A\| \le \gamma_n \|\hat L\|\|\hat U\|$ |

## Литература


1. Голуб Дж., Ван Лоун Ч. Матричные вычисления. — §3.2 (LU-разложение, матрицы Гаусса), §3.4 (выбор ведущего элемента).
2. Higham N. J. Accuracy and Stability of Numerical Algorithms. — гл. 9.
3. Ортега Дж. Введение в параллельные и векторные методы решения линейных систем.

# Этап 1. Последовательное блочное LU-разложение

Программа: stage1_seq.c. Результаты: stage1_seq.csv, график stage1_error.png.

Реализован **Алгоритм 37** из параграфа 2.4 пособия Д. Л. Головашкина «Векторные алгоритмы вычислительной линейной алгебры» (2019, с. 57–62). Этот файл описывает сам алгоритм. Все остальные этапы (векторизация, OpenMP, MPI) выполняют те же вычисления, но по-разному организуют данные и работу.

Математическая теория LU-разложения с полным выводом формул (метод Гаусса в матричной форме, схема Дулиттла, существование и единственность, дополнение Шура, трудоёмкость, устойчивость, разобранный пример) — в отдельном документе.

## 1. Постановка задачи

Дана квадратная матрица $A \in \mathbb{R}^{n\times n}$. Требуется найти разложение

$$A = L\,U,$$

где $L$ — нижняя унитреугольная матрица ($\ell_{ii} = 1$), $U$ — верхняя треугольная. После разложения система $Ax = b$ решается двумя треугольными системами:

$$L\,y = b \quad \text{(прямой ход)}, \qquad U\,x = y \quad \text{(обратный ход)}.$$

### Когда разложение существует

Доказательство этого и следующих утверждений — в теории, раздел 5.

LU-разложение без перестановок строк существует и единственно, если все ведущие главные миноры отличны от нуля:

$$\det A_{1:k,\,1:k} \neq 0, \qquad k = 1, \ldots, n-1.$$

Достаточное условие — строгое диагональное преобладание по строкам или по столбцам, а также симметричная положительная определённость. В программе выбора ведущего элемента **нет**, как и в Алгоритме 37, поэтому тестовая матрица подбирается так, чтобы диагональ была заметно больше остальных элементов (см. раздел 6).

### Хранение на месте

Матрицы $L$ и $U$ записываются поверх $A$. Единичная диагональ $L$ не хранится:

$$A \;\longrightarrow\; \begin{pmatrix} u_{11} & u_{12} & u_{13} & \cdots \\ \ell_{21} & u_{22} & u_{23} & \cdots \\ \ell_{31} & \ell_{32} & u_{33} & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}.$$

Матрица хранится по строкам (row-major): элемент $a_{ij}$ лежит по адресу `A[i*n + j]`; в коде это макрос `A(i, j)`. Индексы в коде начинаются с нуля, в формулах этого раздела — с единицы, как в пособии.

## 2. Вывод алгоритма

Выделим из матрицы первый малый блок размера $\alpha \times \alpha$ и запишем разложение в виде трёх сомножителей (формула (18) пособия):

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11} & 0 \\ L_{21} & E \end{pmatrix} \begin{pmatrix} E & 0 \\ 0 & A' \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ 0 & E \end{pmatrix},$$

где $A_{11}, L_{11}, U_{11} \in \mathbb{R}^{\alpha\times\alpha}$, $A_{21}, L_{21} \in \mathbb{R}^{(n-\alpha)\times\alpha}$, $A_{12}, U_{12} \in \mathbb{R}^{\alpha\times(n-\alpha)}$, $A_{22}, A' \in \mathbb{R}^{(n-\alpha)\times(n-\alpha)}$, а $E$ — единичные матрицы. Первый блочный столбец первого сомножителя — первый блочный столбец $L$, первая блочная строка третьего — первая блочная строка $U$. Перемножив:

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11}U_{11} & L_{11}U_{12} \\ L_{21}U_{11} & L_{21}U_{12} + A' \end{pmatrix},$$

получаем четыре соответствия и порядок их использования:

| Шаг | Равенство | Что ищется | Как |
|---|---|---|---|
| 1 | $A_{11} = L_{11}U_{11}$ | $L_{11}, U_{11}$ | неблочное LU блока $\alpha \times \alpha$ |
| 2 | $A_{21} = L_{21}U_{11}$ | $L_{21}$ | треугольные системы $xU = b$ |
| 3 | $A_{12} = L_{11}U_{12}$ | $U_{12}$ | треугольные системы $Lx = b$ |
| 4 | $A_{22} = L_{21}U_{12} + A'$ | $A'$ | **блочная модификация внешним произведением** |

Матрица $A' = A_{22} - L_{21}U_{12}$ — это дополнение Шура блока $A_{11}$. Осталось разложить её тем же способом: «далее за матрицу $A$ принимается $A'$, и шаги алгоритма повторяются».

После одной итерации матрица хранится так, как на рисунке 8а пособия: $L_{11}\backslash U_{11}$ в левом верхнем блоке, $U_{12}$ справа от него, $L_{21}$ под ним и $A'$ в правом нижнем углу. Программа после каждой итерации даёт ровно такую картину, и произведение трёх сомножителей формулы (18) равно исходной матрице; численная сверка — в пошаговом разборе.

Матрица разбита на $N^2$ малых блоков размера $\alpha = n/N$. После $N - 1$ таких итераций остаётся разложить последний малый блок в правом нижнем углу.

## 3. Шаги алгоритма в поэлементной записи

Обозначим на итерации $k$ границы блоков: $a$ — индексы текущего блока, $b$ — строки малого блока внутри блочного столбца, $c$ — столбцы малого блока внутри блочной строки.

### Шаг 1. LU малого блока

Пособие разрешает здесь «один из ранее сформулированных векторных алгоритмов». При хранении по строкам выгоден Алгоритм 30 со строчным saxpy (kij-форма) — все циклы идут вдоль строки:

$$\ell_{sl} = \frac{a_{sl}}{u_{ll}}, \qquad a_{s,\,l+1:} \;\leftarrow\; a_{s,\,l+1:} - \ell_{sl}\, u_{l,\,l+1:}, \qquad l \in a,\ s > l,\ s \in a.$$

### Шаг 2. Решение $L(b,a)\,U(a,a) = A(b,a)$

Система с верхней треугольной матрицей $U_{11}$ разбивается на набор систем — по одной на каждый малый блок блочного столбца. Каждая строка блока — система $xU = b$ (задача 2.1.3 пособия):

$$x_l = \frac{b_l}{u_{ll}}, \qquad b_{l+1:} \;\leftarrow\; b_{l+1:} - x_l\, u_{l,\,l+1:}, \qquad l \in a.$$

### Шаг 3. Решение $L(a,a)\,U(a,c) = A(a,c)$

Система с нижней унитреугольной матрицей $L_{11}$ разбивается на набор систем — по одной на каждый малый блок блочной строки. Каждый столбец блока — система $Lx = b$, которую решает Алгоритм 27:

$$b_{l+1:} \;\leftarrow\; b_{l+1:} - \ell_{l+1:,\,l}\, b_l, \qquad l \in a.$$

В коде циклы по $s$ и $q$ переставлены: $a_{sq} \mathrel{-}= \ell_{sl}\, u_{lq}$ идёт вдоль строки, что при хранении по строкам обращается к соседним адресам. Операции над каждым столбцом при этом те же и в том же порядке. Деления нет: $\ell_{ll} = 1$.

### Шаг 4. Основная операция

$$\boxed{A'(b,c) = A(b,c) - L(b,a)\,U(a,c)}$$

для всех пар малых блоков $(i,j), i,j > k$. Умножение блочного столбца на блочную строку организовано через произведение малых блоков — ради этого блочный алгоритм и составлялся: каждое такое произведение работает с тремя блоками $\alpha \times \alpha$, которые помещаются в кэш. Новые значения сразу записываются поверх $A$.

### Полный алгоритм

Алгоритм 37 в нумерации с нуля; неполный последний блок (задача 2.4.4) и итерация $k = N - 1$, где из четырёх шагов остаётся только первый, не требуют отдельного кода:

```
N = ⌈n / α⌉
for k = 0 .. N−1:
    a = [kα, min((k+1)α, n))

    # 1. A(a,a) = L(a,a) U(a,a)
    for l in a:
        for s in a, s > l:
            A(s, l) /= A(l, l)
            A(s, l+1 : конец a) −= A(s, l) * A(l, l+1 : конец a)

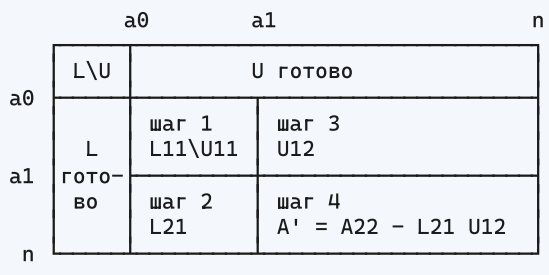
    # 2. L(b,a) U(a,a) = A(b,a)
    for i = k+1 .. N−1:
        for s in блок i:
            for l in a:
                A(s, l) /= A(l, l)
                A(s, l+1 : конец a) −= A(s, l) * A(l, l+1 : конец a)

    # 3. L(a,a) U(a,c) = A(a,c)
    for j = k+1 .. N−1:
        for l in a:
            for s in a, s > l:
                A(s, блок j) −= A(s, l) * A(l, блок j)

    # 4. A'(b,c) = A(b,c) − L(b,a) U(a,c)
    for i = k+1 .. N−1:
        for j = k+1 .. N−1:
            for s in блок i:
                for l in a:
                    A(s, блок j) −= A(s, l) * A(l, блок j)
```

Это ровно функция `block_lu` в stage1_seq.c.

### Схема одной итерации



Каждая итерация обновляет **всю** правую нижнюю подматрицу (right-looking). В left-looking алгоритме, наоборот, столбец не трогается до своей очереди и получает все обновления сразу.

### Упрощённые варианты

В пособии упомянуты ещё два варианта с той же четвёртой операцией: вместо шагов 1–2 выполнить $\alpha$ итераций LU через gaxpy (Алгоритм 34) или вместо шагов 1–3 — $\alpha$ итераций LU через мм. (Алгоритм 30). В программе реализован основной вариант — сам Алгоритм 37.

## 4. Вычислительная сложность

Считаем умножение и вычитание отдельными флопами. На итерации с началом блока $x = k\alpha$ оставшийся размер $m = n - x - \alpha$:

| Шаг | Флопов на итерации | Сумма по итерациям | Доля при $n = 3000$ |
|---|---|---|---|
| 1. LU блока | $\frac{2}{3}\alpha^3$ | $\frac{2}{3}n\alpha^2$ | 0,01% |
| 2. $L_{21}$ | $m\,\alpha^2$ | $\approx \frac{1}{2}n^2\alpha$ | 0,8% |
| 3. $U_{12}$ | $m\,\alpha^2$ | $\approx \frac{1}{2}n^2\alpha$ | 0,8% |
| 4. мм. | $2\,m^2\alpha$ | $\approx \frac{2}{3}n^3 - n^2\alpha$ | 98,4% |

Слагаемые $n^2\alpha$ сокращаются, и итого

$$T_{\text{flops}} \approx \frac{2}{3}\,n^3,$$

что совпадает со сложностью метода Гаусса (ответ на задачу 2.4.3 пособия). Вывод для следующих этапов: **почти вся работа — в шаге 4**, поэтому алгоритм хорошо векторизуется и хорошо параллелится.

Память: $n^2$ чисел для $A$ (разложение на месте). Для проверки программа хранит ещё копию исходной матрицы $A_0$ — ещё $n^2$ чисел.

### Выбор размера блока

В задаче 2.4.5 пособия размер блока выбирается по ёмкости «быстрой» памяти на шаге 4: в кэш должны поместиться три блока $\alpha \times \alpha$ ($A(b,c)$, $L(b,a)$, $U(a,c)$), то есть $3\alpha^2 \le M$, где $M$ — ёмкость кэша в числах. Для кэша L1 процессора i5-1135G7 (48 КБ, $M = 6000$) получаем $\alpha \le 44$.

В программе $\alpha = 32$ (константа `ALPHA`): три блока занимают 24 КБ, а строка блока — 256 байт, ровно 8 векторов AVX2 (этап 2). Пробные замеры при $n = 2000$:

| $\alpha$ | 16 | 32 | 64 | 128 | 256 |
|---|---|---|---|---|---|
| $T$ без векторизации, с | 1,99 | 2,17 | 1,81 | 1,66 | 1,53 |
| $T$ с векторизацией, с | 1,09 | 0,89 | 0,91 | 1,07 | 0,95 |

Измерения однократные и шумные (ноутбук работал от батареи), поэтому таблица показывает лишь порядок. У векторной сборки минимум — при $\alpha = 32$. Скалярная сборка, наоборот, быстрее при больших блоках; причины этого замерами не выделены.

## 5. Решение системы и проверка

После разложения функция `lu_solve` решает $LUx = b$.

**Прямой ход** ($L$ с единичной диагональю):

$$y_i = b_i - \sum_{j<i} \ell_{ij}\, y_j, \qquad i = 0, 1, \dots, n-1.$$

**Обратный ход**:

$$x_i = \frac{1}{u_{ii}} \Big( y_i - \sum_{j>i} u_{ij}\, x_j \Big), \qquad i = n-1, \dots, 0.$$

Оба хода стоят $O(n^2)$ и в замер времени не входят.

## 6. Тестовая матрица и измерение погрешности

### Генерация

$$a_{ij} = \xi_{ij} - \tfrac{1}{2}, \quad \xi_{ij} \sim U[0,1]; \qquad a_{ii} \mathrel{+}= \frac{n}{4}.$$

Генератор каждый раз инициализируется `srand(12345)`, поэтому матрица для данного $n$ всегда одна и та же. Сдвиг диагонали делает её доминирующей **в среднем**, но не гарантирует строгого преобладания: математическое ожидание суммы модулей внедиагональных элементов строки

$$\mathbb{E} \sum_{j \neq i} |a_{ij}| = (n-1) \cdot \mathbb{E}|\xi - \tfrac{1}{2}| = \frac{n-1}{4}$$

почти равно диагонали $n/4 \pm 1/2$. Для этих матриц разложение без выбора ведущего элемента на практике проходит, но это свойство конкретных тестовых данных, а не гарантия.

### Точное решение

Правая часть — сумма элементов каждой строки:

$$b_i = \sum_j a_{ij} \quad \Longrightarrow \quad x^* = (1, 1, \dots, 1)^T.$$

### Метрики

Программа печатает две величины:

$$\mathrm{err} = \|x - x^*\|_\infty = \max_i |x_i - 1|, \qquad \mathrm{res} = \frac{\|A_0 x - b\|_\infty}{\|b\|_\infty}.$$

- `err` — **прямая** погрешность: насколько найденное решение отличается от точного;
- `res` — относительная **невязка**: насколько хорошо найденный $x$ удовлетворяет исходной системе.

### Чего ожидать теоретически

Для метода Гаусса в арифметике с плавающей точкой (машинная точность $\varepsilon = 2^{-53} \approx 1{,}1 \cdot 10^{-16}$) вычисленные множители удовлетворяют

$$\hat{L}\hat{U} = A + \delta A, \qquad |\delta A| \le \gamma_n\, |\hat{L}|\, |\hat{U}|, \qquad \gamma_n = \frac{n\varepsilon}{1 - n\varepsilon},$$

(Higham, теорема 9.3; аналогичная оценка — у Голуба и Ван Лоуна, §3.3). Для прямой погрешности

$$\frac{\|x - x^*\|}{\|x^*\|} \lesssim \kappa(A) \cdot \gamma_n \cdot \rho,$$

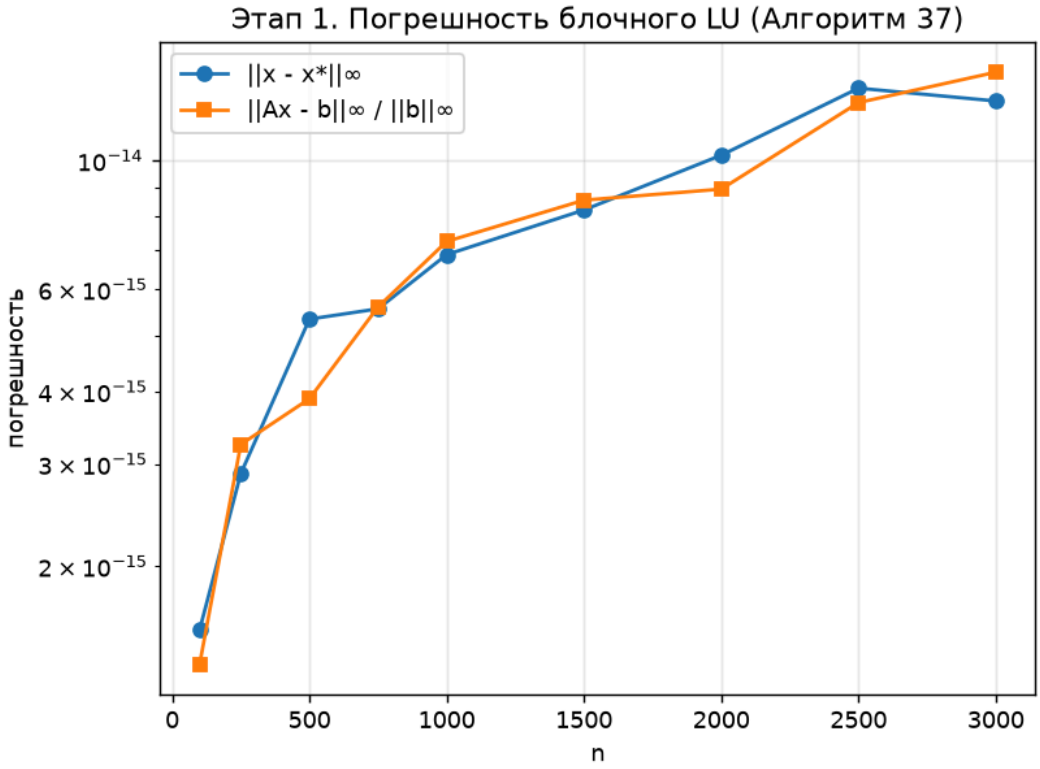
где $\kappa(A) = \|A\|\|A^{-1}\|$ — число обусловленности, $\rho$ — коэффициент роста элементов. Для матриц с выраженной диагональю $\kappa(A)$ и $\rho$ малы, поэтому **ожидается рост погрешности примерно**

**пропорционально** $n\varepsilon$, но с запасом: оценка $\gamma_n$ — худший случай, а на практике ошибки округления частично компенсируются (статистически рост ближе к $\sqrt{n}\,\varepsilon$).

### Результаты

По stage1_seq.csv:

| $n$ | 100 | 250 | 500 | 750 | 1000 | 1500 | 2000 | 2500 | 3000 |
|---|---|---|---|---|---|---|---|---|---|
| err, $10^{-15}$ | 1,55 | 2,89 | 5,33 | 5,55 | 6,88 | 8,22 | 10,21 | 13,32 | 12,66 |
| res, $10^{-15}$ | 1,35 | 3,24 | 3,89 | 5,59 | 7,25 | 8,54 | 8,93 | 12,58 | 14,19 |
| время, с | 0,0002 | 0,003 | 0,025 | 0,086 | 0,207 | 0,707 | 1,72 | 3,39 | 5,88 |




Выводы:

- погрешность **растёт с размерностью**: при увеличении $n$ в 30 раз (с 100 до 3000) `err` растёт примерно в 8 раз — медленнее линейного $n\varepsilon$ и ближе к $\sqrt{n}\,\varepsilon$;
- рост не строго монотонный (при $n = 2500$ ошибка больше, чем при $n = 3000$): каждая матрица своя, и случайный характер ошибок округления заметен;
- во всех случаях ошибка остаётся порядка $10\,\varepsilon$–$100\,\varepsilon$, то есть разложение для этих матриц численно устойчиво;
- время растёт как $n^3$: от $n = 1000$ до $n = 3000$ — в 28,4 раза при теоретических 27.

## 7. Соответствие коду

| Элемент алгоритма | Где в stage1_seq.c |
|---|---|
| Хранение $A$ по строкам | макрос `A(i, j)` |
| Граница малого блока, неполный блок | макрос `END(i)` |
| Размер блока $\alpha$, число повторов | `ALPHA = 32` , `REPS = 3` |
| Шаги 1–4 Алгоритма 37 | `block_lu` |
| Прямой и обратный ход | `lu_solve` |
| Генерация $A_0$, $b$, замер, метрики | `run` |
| Список размеров по умолчанию | `main` |

Замер времени — `QueryPerformanceCounter` на Windows, `clock_gettime` на Linux и macOS; из трёх повторов берётся минимальное время. Копирование $A_0 \to A$ выполняется до запуска таймера.

## Литература


1. Головашкин Д. Л. Векторные алгоритмы вычислительной линейной алгебры. — 2019. — §2.1 (Алгоритм 27, задача 2.1.3), §2.2 (Алгоритм 30), §2.4 (Алгоритм 37).
2. Голуб Дж., Ван Лоун Ч. Матричные вычисления. — §3.2 (формы LU), §3.3 (анализ ошибок).
3. Higham N. J. Accuracy and Stability of Numerical Algorithms. — гл. 9 (LU-разложение), гл. 13 (блочные алгоритмы).
4. Ортега Дж. Введение в параллельные и векторные методы решения линейных систем.

# Этап 2. Векторизация блочного LU-разложения

Программа: stage2_vec.c. Результаты: stage2_novec.csv, stage2_vec.csv, график stage2_vec_speedup.png.

Алгоритм тот же, что на этапе 1: Алгоритм 37 с четырьмя шагами на каждой итерации. Здесь описано, как код подготовлен к автоматической векторизации компилятором GCC и что из этого получилось.

## 1. Что такое векторизация

Современный процессор умеет выполнять одну инструкцию сразу над несколькими числами — **SIMD** (Single Instruction, Multiple Data). На процессоре Intel i5-1135G7 доступен набор **AVX2**: регистры по 256 бит, то есть

$$\frac{256 \text{ бит}}{64 \text{ бита на double}} = 4 \text{ числа double в одном регистре.}$$

Дополнительно набор **FMA** (fused multiply-add) выполняет за одну инструкцию

$$d \leftarrow a \cdot b + c$$

с одним округлением вместо двух. Векторная FMA над 4 числами делает 8 флопов.

**Автовекторизация** — преобразование компилятором скалярного цикла

```
for (q = 0; q < m; q++)
    y[q] -= s * x[q];
```

в цикл, который за одну итерацию обрабатывает 4 элемента:

$$\begin{pmatrix} y_q \\ y_{q+1} \\ y_{q+2} \\ y_{q+3} \end{pmatrix} \leftarrow \begin{pmatrix} y_q \\ y_{q+1} \\ y_{q+2} \\ y_{q+3} \end{pmatrix} - s \cdot \begin{pmatrix} x_q \\ x_{q+1} \\ x_{q+2} \\ x_{q+3} \end{pmatrix}$$ (одна инструкция, например `vfnmadd231pd`).

Теоретическое ускорение такого цикла — до 4 раз за счёт ширины вектора (и ещё до 2 раз, если без FMA умножение и вычитание выполнялись бы отдельно). На практике оно ограничено скоростью подачи данных из памяти.

### Условия, при которых компилятор векторизует цикл

1. **Нет зависимостей между итерациями** внутреннего цикла: итерация $q$ не читает то, что пишет итерация $q' \neq q$.
2. **Единичный шаг по памяти**: элементы $x_q, x_{q+1}, \ldots$ лежат подряд — тогда их можно загрузить одной векторной инструкцией.
3. **Известно отсутствие перекрытия** (aliasing) массивов $y$ и $x$. Если компилятор не может это доказать, он либо отказывается от векторизации, либо вставляет проверку во время выполнения (*versioning*).
4. Простая форма цикла: счётчик с известными границами, без вызовов функций и ветвлений в теле.

## 2. Все шаги алгоритма — одна операция saxpy

Самая внутренняя операция всех четырёх шагов Алгоритма 37 одинакова и идёт вдоль строки:

$$y \leftarrow y - s \cdot x, \qquad a_{sq} \mathrel{-}= a_{sl}\, a_{lq},$$

| Шаг | $y$ | $s$ | $x$ | Длина |
|---|---|---|---|---|
| 1. LU блока | строка $s$ блока | $\ell_{sl}$ | строка $l$ блока | $< \alpha$ |
| 2. $L_{21}$ | строка $s$ столбца $L$ | $\ell_{sl}$ | строка $l$ блока $U_{11}$ | $< \alpha$ |
| 3. $U_{12}$ | строка $s$ блока $U(a,c)$ | $\ell_{sl}$ | строка $l$ блока $U(a,c)$ | $\alpha$ |
| 4. мм. | строка $s$ блока $A(b,c)$ | $\ell_{sl}$ | строка $l$ блока $U(a,c)$ | $\alpha$ |

Поэтому в stage2_vec.c вся арифметика разложения вынесена в одну функцию:

```
static inline void saxpy(int m, double s, const double *restrict x, double *restrict y)
{
    for (int q = 0; q < m; q++)
        y[q] -= s * x[q];
}
```

Условия 1, 2 и 4 выполнены: при хранении по строкам $x$ и $y$ — соседние элементы строки матрицы. Условие 3 обеспечивает ключевое слово `restrict`: это обещание программиста, что через `y` нельзя изменить данные, которые читаются через `x`. Обещание верно, так как $x$ и $y$ — **разные строки** матрицы ($s > l$ или $s$ лежит ниже блока $a$).

Именно поэтому в шаге 1 применена kij-форма LU, а в шаге 3 циклы переставлены по сравнению с Алгоритмом 27: при хранении по строкам внутренний цикл вдоль столбца шёл бы с шагом $n$, и его нельзя было бы загрузить одной векторной инструкцией.

На этапе 1 та же арифметика записана через `A(s, q) -= A(s, l) * A(l, q)`: все обращения идут к одному массиву `A` без `restrict`, и компилятору сложнее доказать, что они не пересекаются.

## 3. Сборки и флаги компилятора

Один исходник собирается дважды (build.sh):

| Сборка | Флаги | Назначение |
|---|---|---|
| `stage2_novec.exe` | `-O2 -fno-tree-vectorize` | база: векторизация запрещена |
| `stage2_vec.exe` | `-O3 -ftree-vectorize -mavx2 -mfma` | векторизация с AVX2 и FMA |

Общие флаги обеих сборок: `-std=gnu99 -msse2 -mfpmath=sse`. Компилятор — 32-битный MinGW GCC 9.2; без `-mfpmath=sse` он использовал бы для `double` устаревший стековый сопроцессор x87, и сравнение было бы нечестным.

Значение флагов:

- `-ftree-vectorize` — включает автовекторизатор циклов (на `-O3` он включён и так; флаг указан явно, чтобы было видно, что именно сравнивается);
- `-fno-tree-vectorize` — явно запрещает его в базовой сборке;
- `-mavx2` — разрешает инструкции AVX2 (256-битные векторы);
- `-mfma` — разрешает инструкции fused multiply-add.

### Отчёт компилятора

Флаг `-fopt-info-vec-optimized` печатает, какие циклы векторизованы. Для `stage2_vec.c` GCC 9.2 сообщает:

```
stage2_vec.c:36:5: optimized: loop vectorized using 32 byte vectors
stage2_vec.c:87:5: optimized: loop vectorized using 32 byte vectors
stage2_vec.c:107:9: optimized: loop vectorized using 32 byte vectors
stage2_vec.c:126:9: optimized: loop vectorized using 32 byte vectors
```

- строка 36 — цикл `saxpy`: векторизован 32-байтными (AVX) векторами без проверок перекрытия, потому что указатели объявлены `restrict`. Это и есть основная работа алгоритма;
- строка 87 — цикл прямого хода в `lu_solve`, вне замера;
- строки 107 и 126 — суммы при вычислении $b$ и невязки, вне замера.

При $\alpha = 32$ цикл `saxpy` на шагах 3–4 — ровно 8 векторных итераций без «хвоста». В шагах 1–2 длина цикла меньше $\alpha$ и зависит от $l$, поэтому там часть элементов выполняется скалярно.

## 4. Модель ожидаемого ускорения

Пусть $T_{\text{scal}}$ — время без векторизации. Разделим его на долю $f$, приходящуюся на векторизуемые циклы, и долю $1 - f$ (все остальные операции). Если векторизуемая часть ускоряется в $s$ раз, то по закону Амдала

$$S = \frac{T_{\text{scal}}}{T_{\text{vec}}} = \frac{1}{(1 - f) + f/s}.$$

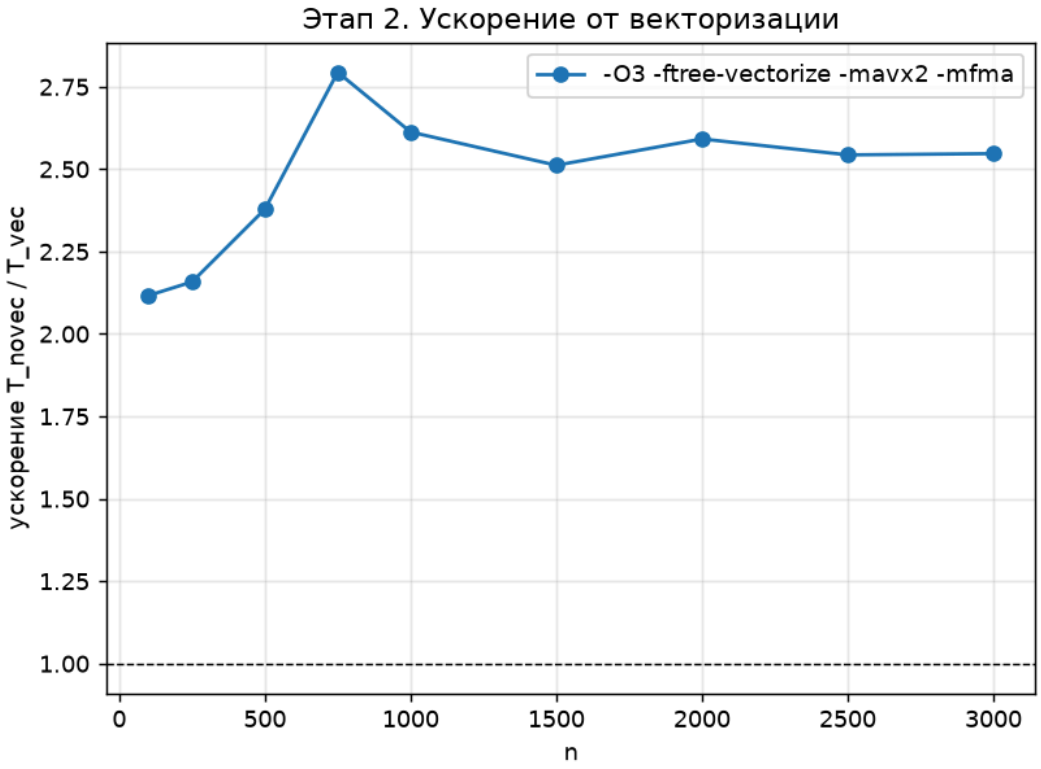
Вся арифметика проходит через `saxpy`, кроме делений в шагах 1 и 2 — их $O(n^2)$ при $\frac{2}{3}n^3$ флопов (раздел 4 этапа 1). Значит $f \approx 1$, и ускорение определяется тем, насколько быстро работает векторный цикл.

Верхняя граница $s$ по арифметике — 4 (ширина AVX2). На одну векторную FMA приходится загрузка 4 элементов $x$ (32 байта) и загрузка с записью 4 элементов $y$. Блоки $x$ и $y$ размером $\alpha \times \alpha$ помещаются в кэш L1 (48 КБ), однако каждая итерация алгоритма заново читает и пишет всю оставшуюся подматрицу ($n = 3000$ — до 72 МБ), и блоки приходится сначала подгружать из памяти. Поэтому реальное $s$ меньше 4.

## 5. Результаты

$$S(n) = \frac{T_{\text{novec}}(n)}{T_{\text{vec}}(n)}$$

| $n$ | 100 | 250 | 500 | 750 | 1000 | 1500 | 2000 | 2500 | 3000 |
|---|---|---|---|---|---|---|---|---|---|
| $T_{\text{novec}}$, с | 0,0002 | 0,003 | 0,030 | 0,109 | 0,257 | 0,850 | 2,04 | 4,01 | 6,90 |
| $T_{\text{vec}}$, с | 0,0001 | 0,002 | 0,012 | 0,039 | 0,098 | 0,338 | 0,786 | 1,58 | 2,71 |
| $S$ | 2,12 | 2,16 | 2,38 | 2,79 | 2,61 | 2,51 | 2,59 | 2,54 | 2,55 |




Выводы:

- при $n \geq 750$ ускорение стабильно **2,5–2,8** и не падает с ростом $n$. Это согласуется с тем, что основная операция работает с блоками $32 \times 32$, которые помещаются в кэш L1;
- при малых $n$ (100–500) ускорение ниже, **2,1–2,4**: блоков мало, и заметную долю времени занимают шаги 1–2 с короткими циклами, где векторные итерации не окупаются;
- до теоретического предела 4 ускорение не доходит. Наиболее вероятная причина — каждая итерация заново загружает блоки оставшейся подматрицы из памяти; отдельно это не измерялось;
- `err` векторной версии совпадает с этапом 1 до последнего знака, кроме $n = 2000$ и $n = 3000$ ($1{,}243 \cdot 10^{-14}$ против $1{,}266 \cdot 10^{-14}$), `res` отличается в большинстве точек: FMA выполняет $a \cdot b + c$ с одним округлением вместо двух, поэтому результаты отличаются на уровне машинной точности.

### Ограничения эксперимента

Сборки отличаются не только векторизацией: база собрана с `-O2`, векторная — с `-O3`. Уровень `-O3` включает и другие оптимизации (например, более агрессивную развёртку циклов). Поэтому измерено ускорение **сборки в целом**; отдельный вклад SIMD, FMA и `-O3` эти замеры не разделяют. Чтобы его выделить, нужны дополнительные сборки, например `-O3 -fno-tree-vectorize` и `-O3 -ftree-vectorize` без `-mfma`.

## 6. Отличия от этапа 1

| | Этап 1 | Этап 2 |
|---|---|---|
| Доступ к элементам | макрос `A(i, j)` | указатели на строки, функция `saxpy` |
| Перекрытие указателей | неизвестно компилятору | `restrict` |
| Множитель $\ell_{sl}$ | читается из памяти на каждой итерации | передан по значению |
| Дополнительная память | нет | нет |

Алгоритм и порядок операций над каждым элементом одинаковы, поэтому `stage2_novec` даёт ту же погрешность, что `stage1_seq`, с точностью до последнего знака. По времени `stage2_novec` при $n \geq 1000$ проигрывает `stage1_seq` примерно 20 % (см. CSV), хотя обе собраны с одними флагами. Цикл `saxpy` встроен, вызовов функции нет; вероятная причина — кодогенерация 32-битного компилятора для этих двух форм записи, отдельно она не исследовалась. На сравнение внутри этапа 2 это не влияет.

# Этап 3. OpenMP: блочное LU-разложение

Программа: stage3_omp.c. Результаты: `results/stage3_omp_t1.csv` … stage3_omp_t8.csv, графики stage3_omp_speedup_p.png и stage3_omp_speedup_n.png.

Алгоритм — Алгоритм 37 из этапа 1, данные и операция `saxpy` организованы как на этапе 2. Параллельные циклы помечены директивой `#pragma omp parallel for`. **Итерации между потоками распределяет сама OpenMP** — в программе нет ручного деления диапазонов по номеру потока (`omp_get_thread_num`), нет явных барьеров и критических секций.

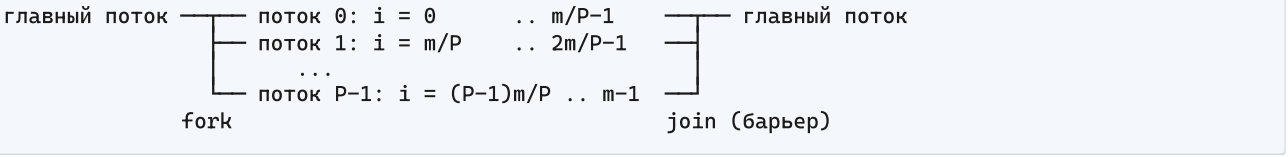
## 1. Модель OpenMP

OpenMP — стандарт параллельного программирования для систем с **общей памятью**. Программа начинается одним (главным) потоком. Директива

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < m; i++)
    body(i);
```

работает по схеме **fork–join**:

1. *fork* — главный поток запускает команду из $P$ потоков (потоки создаются один раз и затем переиспользуются библиотекой libgomp);
2. итерации цикла $0, \dots, m-1$ распределяются между потоками;
3. каждый поток выполняет свои итерации;
4. *join* — неявный барьер в конце цикла: главный поток продолжает работу, когда все итерации завершены.




### Распределение итераций

`schedule(static)` — итерации делятся на $P$ непрерывных частей почти равного размера. Стандарт OpenMP оставляет точное деление реализации; в libgomp (GCC) при $q = \lfloor m/P \rfloor$ и $r = m \bmod P$ поток $t$ получает

$$\big[\, tq + \min(t, r),\;\; (t+1)q + \min(t+1, r) \,\big), \qquad t = 0, \dots, P-1,$$

то есть первые $r$ потоков — на одну итерацию больше. Например, 25 итераций на 3 потока делятся как $9 + 8 + 8$ (это видно в трассировке docs/trace/stage3_threads_n12_p3.txt).

Это деление выполняет библиотека OpenMP, а не программа. Статическое расписание подходит, потому что все итерации каждого параллельного цикла имеют почти одинаковую стоимость: малые блоки одного размера.

### Переменные

Переменные, объявленные **внутри** тела цикла (`c0`, `m`, `s`, `l`, …), и счётчик параллельного цикла — **локальные** для каждого потока. Массив `A` — **общий**: корректность обеспечивается тем, что разные итерации пишут в разные элементы (см. раздел 2).

### Число потоков

Число потоков задаётся при запуске переменной окружения:

```
OMP_NUM_THREADS=4 ./bin/stage3_omp.exe 1000 2000
```

Программа печатает его в столбце `threads` (через `omp_get_max_threads()`). Сборка одна:

```
gcc -std=gnu99 -msse2 -mfpmath=sse -O2 -fno-tree-vectorize -fopenmp stage3_omp.c -o
bin/stage3_omp.exe
```

Флаг `-fopenmp` включает обработку директив и подключает libgomp. Векторизация запрещена (`-fno-tree-vectorize`), как и на этапе 1, — чтобы измерить вклад именно многопоточности. База для ускорения — та же программа с `OMP_NUM_THREADS=1`.

## 2. Какие циклы можно распараллелить

Директива `parallel for` корректна, только если итерации цикла **независимы**: ни одна итерация не читает и не пишет то, что пишет другая. Различают зависимости:

| Тип | Итерация $i_1$ | Итерация $i_2 > i_1$ | Пример |
|---|---|---|---|
| истинная (flow, RAW) | пишет | читает | `x[i] = x[i-1] + 1` |
| анти- (WAR) | читает | пишет | `x[i] = x[i+1]` |
| по выходу (WAW) | пишет | пишет | `s = x[i]` |

Компилятор при `#pragma omp parallel for` **не проверяет** независимость — за неё отвечает программист. Разберём четыре шага алгоритма. Внешний цикл по $k$ параллелить нельзя: итерация $k$ работает с матрицей $A'$, полученной на итерации $k-1$.

### Шаг 1: LU малого блока — последовательно

$$a_{sq} \mathrel{-}= \ell_{sl}\, u_{lq}, \qquad s, q \in a,\ s, q > l.$$

Строки блока зависят друг от друга: строка $l$ должна быть готова раньше строк $s > l$. На шаг приходится $0{,}01\%$ работы при $n = 3000$, поэтому он оставлен последовательным.

### Шаг 2: $L(b,a)\,U(a,a) = A(b,a)$ — блоки блочного столбца независимы

Итерация $i$ **пишет** только строки своего малого блока $b$ в столбцах $a$, а **читает** их же и блок $U(a,a)$, который на этом шаге не меняется:

```
#pragma omp parallel for schedule(static)
for (int i = k + 1; i < N; i++)
    for (int s = i * alpha; s < END(i); s++)
        for (int l = a0; l < a1; l++) {
            ROW(s)[l] /= ROW(l)[l];
            saxpy(a1 - l - 1, ROW(s)[l], ROW(l) + l + 1, ROW(s) + l + 1);
        }
```

### Шаг 3: $L(a,a)\,U(a,c) = A(a,c)$ — блоки блочной строки независимы

Внутри одного малого блока строки зависимы: строка $s$ использует строки $l < s$. Но система $L_{11}U_{12} = A_{12}$ разбивается на набор систем с разными правыми частями, и малые блоки $U(a,c)$ не зависят друг от друга. Итерация $j$ пишет только столбцы своего блока $c$:

```
#pragma omp parallel for schedule(static)
for (int j = k + 1; j < N; j++) {
    int c0 = j * alpha, m = END(j) - c0;
    for (int l = a0; l < a1; l++)
        for (int s = l + 1; s < a1; s++)
            saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
}
```

Разные потоки пишут в одни и те же строки матрицы, но в разные столбцы. На границе малых блоков два потока могут работать с одной кэш-линией — это ложное разделение (false sharing), но оно затрагивает лишь края блоков длиной 256 байт, и шаг занимает меньше $1\%$ работы.

### Шаг 4: основная операция — пары блоков независимы

Итерация $(i, j)$ **пишет** только блок $A(b, c)$, а **читает** его же, $L(b, a)$ и $U(a, c)$. Блоки $L(b, a)$ и $U(a, c)$ лежат в столбцах $a$ и строках $a$, куда на этом шаге никто не пишет. Значит, две итерации пересекаются по записи, только если совпадают и $i$, и $j$:

```
#pragma omp parallel for collapse(2) schedule(static)
for (int i = k + 1; i < N; i++)
    for (int j = k + 1; j < N; j++) {
        int c0 = j * alpha, m = END(j) - c0;
        for (int s = i * alpha; s < END(i); s++)
            for (int l = a0; l < a1; l++)
                saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
    }
```

`collapse(2)` объединяет два цикла в один из $(N - k - 1)^2$ итераций, и OpenMP делит на потоки уже его. Это даёт хорошую балансировку и на последних итерациях, когда блоков остаётся меньше, чем потоков. Порядок вычитаний над каждым элементом ($l$ по возрастанию) не меняется.

### Сводка

| Шаг | Как выполняется | Число итераций | Флопы (всего) |
|---|---|---|---|
| 1. LU блока | последовательно | — | $\frac{2}{3}n\alpha^2$ |
| 2. $L_{21}$ | параллельно по блокам столбца | $N - k - 1$ | $\approx \frac{1}{2}n^2\alpha$ |
| 3. $U_{12}$ | параллельно по блокам строки | $N - k - 1$ | $\approx \frac{1}{2}n^2\alpha$ |
| 4. мм. | параллельно по парам блоков | $(N - k - 1)^2$ | $\approx \frac{2}{3}n^3 - n^2\alpha$ |

На одну итерацию по $k$ приходится **три** параллельные области, при $n = 3000$ — около 280 за всё разложение.

## 3. Модель ускорения

Обозначим $T_1 \approx \frac{2}{3}n^3 t_c$ — время на одном потоке. Последовательная часть — только шаг 1. Для неё по закону Амдала

$$\sigma = \frac{\frac{2}{3}n\alpha^2}{\frac{2}{3}n^3} = \frac{\alpha^2}{n^2}, \qquad S(P) = \frac{T_1}{T_P} \approx \frac{1}{\sigma + (1 - \sigma)/P}.$$

При $n = 3000$ и $\alpha = 32$ доля $\sigma \approx 10^{-4}$, поэтому модель даёт $S(P) \approx P$: алгоритм с модификацией внешним произведением почти идеально параллелится по арифметике.

Модель не учитывает:

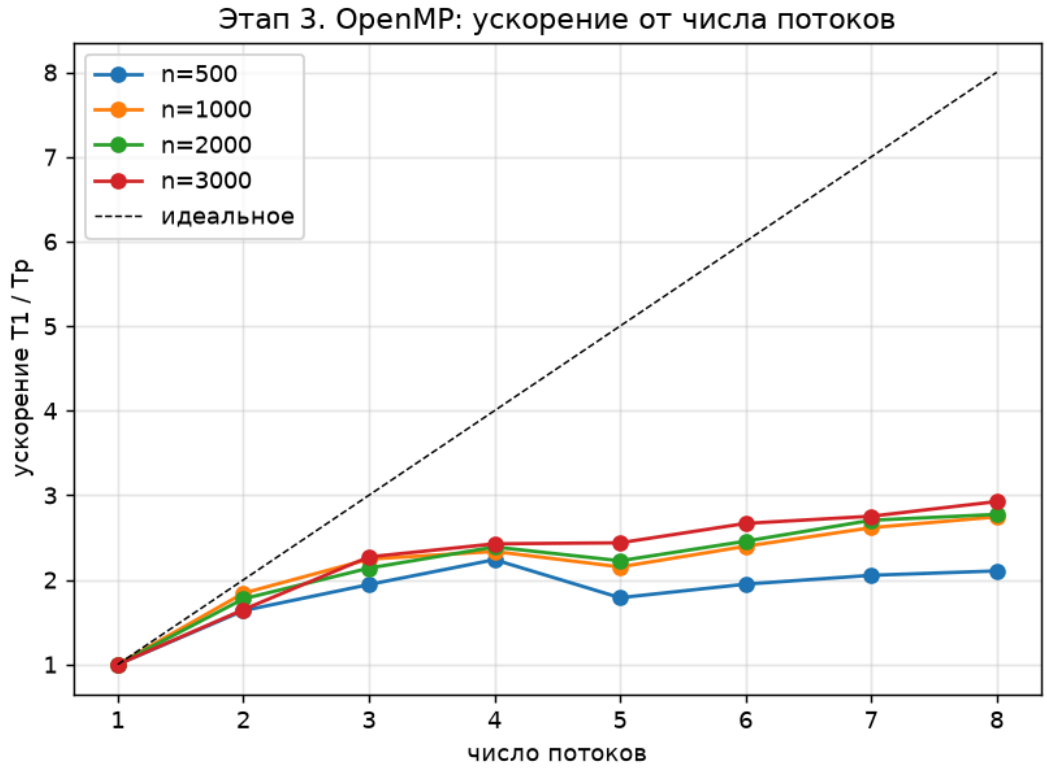
- **общую пропускную способность памяти.** Каждая итерация по $k$ читает и записывает всю оставшуюся подматрицу, а при $n = 3000$ это до 72 МБ — больше кэша L3 (8 МБ). Несколько потоков одновременно упираются в канал памяти;
- **снижение частоты.** При загрузке всех ядер Turbo Boost снижает частоту, поэтому один поток работает быстрее, чем каждый из четырёх;
- **накладные расходы fork–join.** Каждая область стоит единицы микросекунд, что заметно только при $n = 100$–$250$;
- **число физических ядер — четыре.** При $P > 4$ потоки делят ядра через Hyper-Threading.

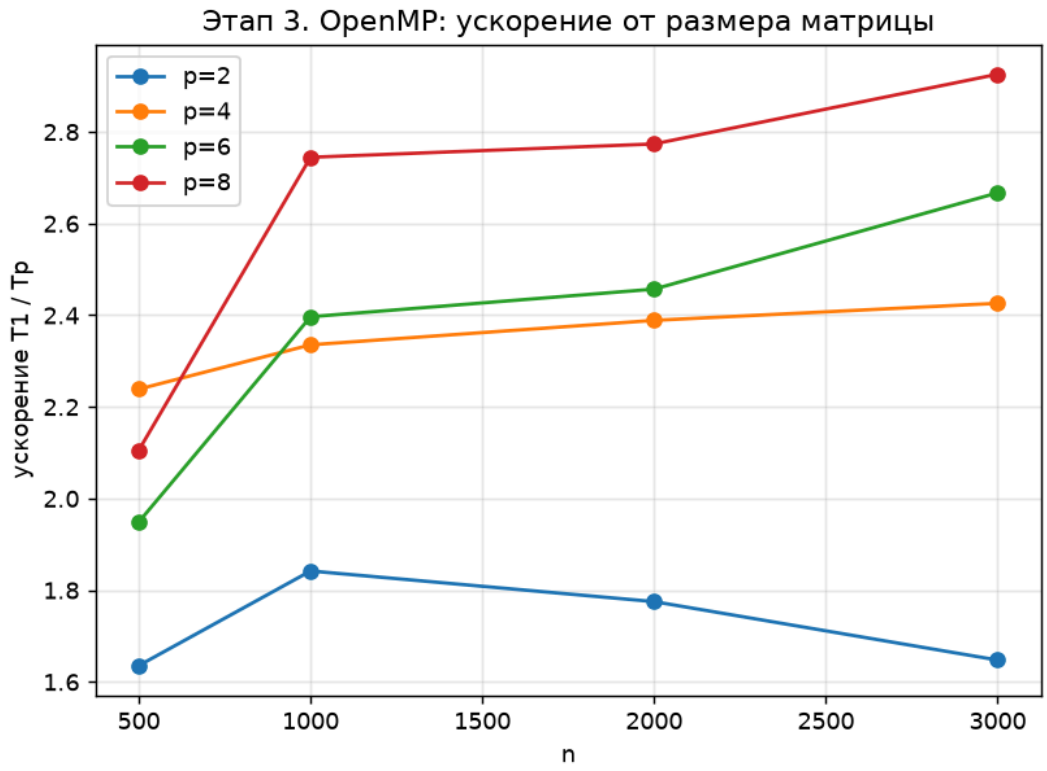
## 4. Результаты

$$S(P) = \frac{T(\texttt{OMP_NUM_THREADS} = 1)}{T(\texttt{OMP_NUM_THREADS} = P)}$$

| $n \setminus P$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| 500 | 1,00 | 1,64 | 1,94 | **2,24** | 1,79 | 1,95 | 2,05 | 2,11 |
| 1000 | 1,00 | 1,84 | 2,24 | 2,34 | 2,15 | 2,40 | 2,62 | **2,74** |
| 2000 | 1,00 | 1,78 | 2,14 | 2,39 | 2,23 | 2,46 | 2,70 | **2,77** |
| 3000 | 1,00 | 1,65 | 2,27 | 2,43 | 2,44 | 2,67 | 2,75 | **2,93** |

Время одного потока: 0,030 с ($n = 500$), 0,256 с ($n = 1000$), 2,010 с ($n = 2000$), 6,892 с ($n = 3000$). Эффективность $E = S/P$ при $n = 3000$: 0,82 ($P = 2$), 0,61 ($P = 4$), 0,37 ($P = 8$).






### Выводы

- при $P = 4$ (число физических ядер) ускорение составляет **2,24–2,43** и растёт с размером матрицы;
- лучшее значение — **2,93** при $n = 3000$, $P = 8$. Потоки сверх четырёх работают на тех же ядрах через Hyper-Threading, поэтому прирост от $P = 4$ до $P = 8$ небольшой;
- при $P = 2$ ускорение 1,64–1,84 и слабо зависит от $n$. При $P = 5$ и $n \leq 2000$ заметен провал: потоков уже больше, чем ядер, и статическое распределение, вероятно, ждёт самый медленный из них;
- реальное ускорение заметно ниже модели $S \approx P$. Доля последовательного кода здесь ничтожна, поэтому ограничение — в оборудовании: общая память, снижение частоты при загрузке всех ядер. Замеры сделаны на ноутбуке от батареи, что усиливает этот эффект и добавляет шум;
- погрешности `err` и `res` для всех $P$ совпадают с последовательной версией: каждый элемент вычисляется одним потоком в том же порядке вычитаний.

### Сравнение с MPI

OpenMP-версия на этом компьютере работает быстрее MPI-версии (этап 4): при $n = 3000$ ускорение 2,43 против 2,18 при четырёх исполнителях и 2,93 против 2,36 при восьми. Потоки обращаются к одной общей матрице без упаковки блочной строки и рассылок. Кроме того, в OpenMP-версии шаги 2–3 делятся на всех, а в MPI шаги 1 и 3 выполняет один процесс, пока остальные ждут рассылку.

# Этап 4. MPI-версия блочного LU-разложения

Программа: stage4_mpi.c. Результаты: `results/stage4_mpi_p1.csv` … stage4_mpi_p8.csv, графики stage4_mpi_speedup_p.png и stage4_mpi_speedup_n.png.

Алгоритм — Алгоритм 37 из этапа 1. Здесь описано, как данные и работа распределяются между $p$ MPI-процессами и какими сообщениями они обмениваются.

## 1. Модель MPI

MPI-программа запускается как $p$ независимых процессов с **раздельной памятью**:

```
mpiexec -n 4 bin/stage4_mpi.exe 1000 2000
```

Каждый процесс знает свой номер `rank` $\in \{0, \dots, p-1\}$ и общее число `size` $= p$. Данные между процессами передаются только явными сообщениями:

| Операция | Что делает | Где |
|---|---|---|
| `MPI_Bcast` | владелец рассылает блочную строку всем | внутри разложения, на каждой итерации |
| `MPI_Send` , `MPI_Recv` | процесс 0 раздаёт и собирает блочные строки | вне замера |
| `MPI_Barrier` | синхронизация всех процессов | вокруг замера |

Используется реализация MS-MPI; все процессы работают на одном компьютере, и сообщения передаются через общую память.

## 2. Распределение данных

Каждый процесс хранит **только свои блочные строки**. Распределение циклическое:

$$\text{владелец блока } i = i \bmod p, \qquad i = 0, \dots, N-1.$$

Например, для $N = 10$ и $p = 3$:

| Процесс | Блочные строки |
|---|---|
| 0 | 0, 3, 6, 9 |
| 1 | 1, 4, 7 |
| 2 | 2, 5, 8 |

Строка хранится целиком ($n$ чисел), потому что в ней лежат и найденные элементы $L$, и ещё не обработанные элементы $A'$. Строки процесса идут в локальном массиве подряд, блок $i$ начинается со строки $\lfloor i/p \rfloor \cdot \alpha$: все предшествующие блоки этого процесса полные, короче может быть только последний блок матрицы.

**Почему циклически.** На итерации $k$ в работе участвуют только блоки $i > k$. При распределении непрерывными кусками процесс с первыми строками быстро остался бы без работы, а при циклическом у всех процессов остаётся поровну блоков почти до конца разложения.

**Почему по строкам.** Шаги 1 и 3 Алгоритма 37 работают с блочной строкой $k$, которая целиком лежит у одного процесса, и не требуют обменов. Шаги 2 и 4 для блока $i$ пишут только в строки блока $i$. Единственное, что нужно снаружи, — блочная строка $k$.

Память на процесс — $\approx n^2/p$ чисел для строк плюс $\alpha n$ чисел для буфера `W`. При $n = 3000$ и $p = 4$ это около 19 МБ вместо 72 МБ. Процесс 0 ещё хранит $A_0$ и собранное разложение для проверки, но это вне алгоритма.

## 3. Параллельный алгоритм одной итерации

Обозначения, как на этапе 1: $a$ — текущий малый блок, $w$ — его размер, $t = n - k\alpha$ — длина строк блочной строки $k$, начиная с диагонального блока.

```
для k = 0 .. N-1:
    q = k mod p                                  # владелец блочной строки k
    если rank == q:
        шаг 1: LU блока A(a,a)                   # локально
        шаг 3: L(a,a) U(a,c) = A(a,c) для всех c  # локально
        упаковать блочную строку k в W           # w x t чисел
    если k == N-1: конец
    Bcast(W, w*t, от q)                          # единственный обмен
    для своих блоков i > k:
        шаг 2: L(b,a) W(0:w, 0:w) = A(b,a)
        шаг 4: A(b,c) -= L(b,a) W(0:w, c) для всех c
```

### Шаги 1 и 3: у владельца

Владелец блочной строки $k$ раскладывает блок и находит $U_{12}$, как на этапе 2, но в координатах локального массива. Затем копирует строки в плотный буфер `W`: у них шаг $n$ в локальном массиве, а для `MPI_Bcast` нужен непрерывный участок. В `W` первые $w$ столбцов — блок $L_{11} \backslash U_{11}$, остальные — $U_{12}$.

### Обмен

`MPI_Bcast` передаёт $w \cdot t$ чисел от владельца всем остальным. После вызова у каждого процесса есть всё, что нужно для шагов 2 и 4. Последний блок рассылать незачем: строк ниже нет.

### Шаги 2 и 4: у всех, над своими строками

Первый свой блок ниже $k$ находится как

$$i_0 = k + 1 + \big((\text{rank} - k - 1) \bmod p\big),$$

далее $i = i_0, i_0 + p, i_0 + 2p, \ldots$. Для каждого блока строки решается система $x\,U_{11} = b$, где $U_{11}$ берётся из `W`, а затем выполняется основная операция по малым блокам $c$:

$$A(b, c) \leftarrow A(b, c) - L(b, a)\, W(0{:}w,\ c).$$

Разные процессы не пересекаются по записи: каждый пишет только в свои строки.

### Совпадение результатов

Каждый элемент вычисляется одним процессом в том же порядке вычитаний, что и на этапе 1, а рассылка копирует числа без изменения. Поэтому `err` и `res` совпадают с этапом 1 до последнего знака при любом $p$.

## 4. Анализ стоимости

Обозначим $t_c$ — время одного флопа, $\lambda$ — задержку сообщения, $\beta$ — время передачи одного байта. Для `MPI_Bcast` объёмом $V$ байт по двоичному дереву

$$T_{\text{bcast}}(V) \approx \lceil \log_2 p \rceil\, (\lambda + V\beta).$$

### Вычисления

| Шаг | Кто выполняет | Сумма по итерациям |
|---|---|---|
| 1. LU блока | один процесс | $\frac{2}{3}n\alpha^2$ |
| 3. $U_{12}$ | один процесс | $\approx \frac{1}{2}n^2\alpha$ |
| 2. $L_{21}$ | все, свои строки | $\approx \frac{1}{2p}n^2\alpha$ |
| 4. мм. | все, свои строки | $\approx \frac{1}{p}\left(\frac{2}{3}n^3 - n^2\alpha\right)$ |

### Обмены

На итерации $k$ рассылается $w \cdot t$ чисел: от $\alpha n$ в начале до почти нуля в конце. За всё разложение — $\sum_k \alpha\,(n - k\alpha) \approx \frac{1}{2}n^2$ чисел, то есть

$$V_{\text{всего}} \approx 4n^2 \text{ байт}, \qquad T_{\text{comm}} \approx \lceil \log_2 p \rceil \left( N\lambda + 4n^2\beta \right).$$

При $n = 3000$ это 36 МБ за разложение и 93 рассылки.

### Итоговая модель

$$T_p \approx \underbrace{\frac{2n^3}{3p}\,t_c}_{\text{делится}} + \underbrace{\frac{n^2\alpha}{2}\,t_c}_{\text{шаг 3, у владельца}} + \underbrace{\lceil \log_2 p \rceil \left( N\lambda + 4n^2\beta \right)}_{\text{обмены}}, \qquad S(p) = \frac{T_1}{T_p}.$$

Главный член растёт как $n^3$, а обмены и неделимая часть — как $n^2$. Поэтому **при росте $n$ ускорение должно приближаться к $p$**, а при малых $n$ и больших $p$ обмены заметнее.

### Балансировка нагрузки

При циклическом распределении число блоков ниже $k$ у разных процессов различается не больше чем на один. Различие заметно лишь в конце разложения, когда блоков ниже $k$ меньше $p$ и работы уже мало.

### Что модель не учитывает

Модель считает, что флоп стоит одинаково независимо от числа процессов. На одном ноутбуке это не так:

- **Пропускная способность памяти общая.** Процессы хранят разные строки, но обращаются к ним через один канал памяти.
- **Частота процессора.** При загрузке всех ядер Turbo Boost снижает частоту.
- **Физических ядер четыре.** При $p > 4$ два процесса делят одно ядро (Hyper-Threading); к тому же MS-MPI в коллективных операциях ожидает данные активным опросом, отнимая у соседнего процесса время ядра.

## 5. Замер времени

```
scatter_rows(n, alpha, A0, A);       /* вне замера */
MPI_Barrier(MPI_COMM_WORLD);
double t = MPI_Wtime();
block_lu_mpi(n, alpha, A);           /* все обмены внутри */
MPI_Barrier(MPI_COMM_WORLD);         /* ждём самого медленного */
t = MPI_Wtime() - t;
```

В замер входят вычисления, все `MPI_Bcast`, упаковка в `W`, выделение буфера и завершающий барьер. Начальная раздача строк и сбор разложения на процессе 0 не входят. Из трёх повторов берётся минимум;

печатает результат процесс 0, он же проверяет решение ( `err` , `res` — как на этапе 1).

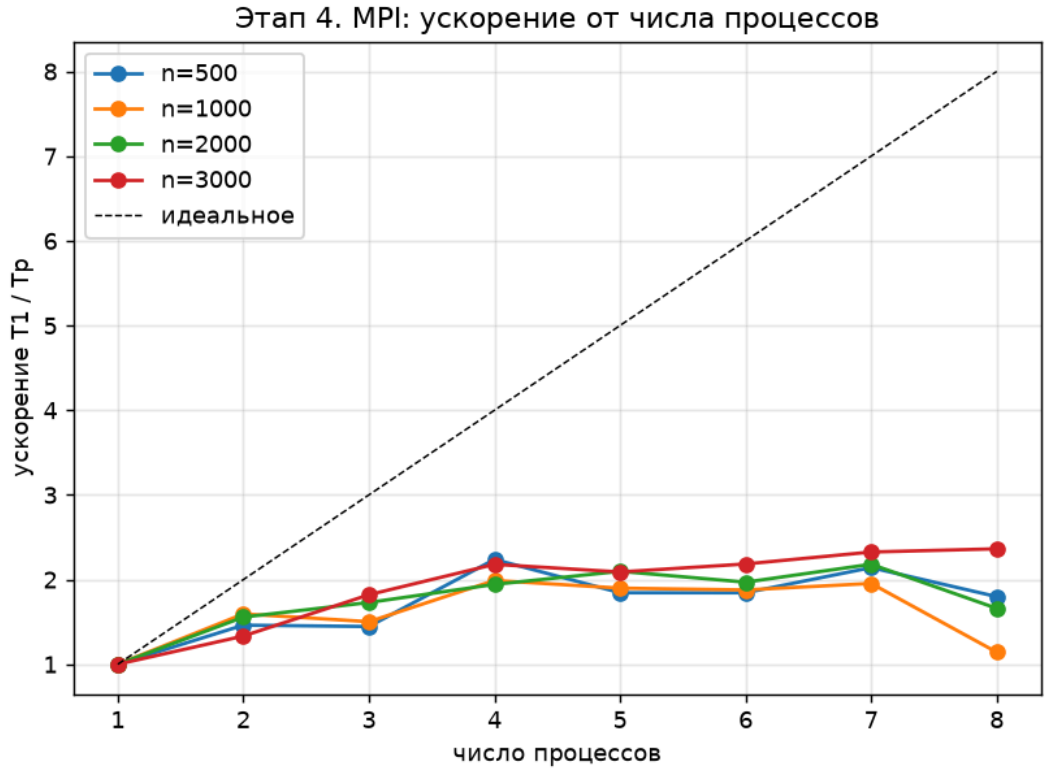
## 6. Результаты

### Ускорение от числа процессов

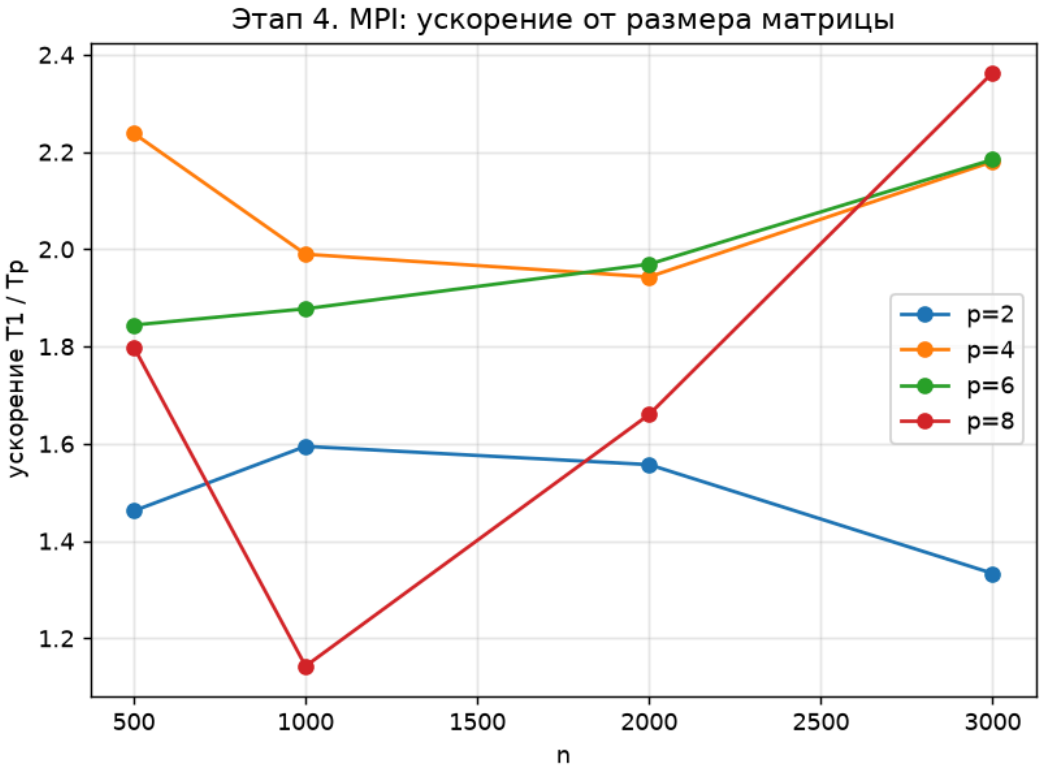
$$S(p) = \frac{T_1}{T_p}$$

| $n \setminus p$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| 500 | 1,00 | 1,46 | 1,45 | **2,24** | 1,84 | 1,84 | 2,14 | 1,80 |
| 1000 | 1,00 | 1,59 | 1,50 | **1,99** | 1,90 | 1,88 | 1,95 | 1,14 |
| 2000 | 1,00 | 1,56 | 1,73 | 1,94 | 2,10 | 1,97 | **2,18** | 1,66 |
| 3000 | 1,00 | 1,33 | 1,82 | 2,18 | 2,09 | 2,18 | 2,33 | **2,36** |

Время одного процесса: 0,033 с ($n = 500$), 0,267 с ($n = 1000$), 2,084 с ($n = 2000$), 7,162 с ($n = 3000$). Эффективность $E = S/p$ при $n = 3000$: 0,67 ($p = 2$), 0,55 ($p = 4$), 0,30 ($p = 8$).



### Ускорение от размера матрицы




### Выводы

- лучшее ускорение — **2,36** при $n = 3000$, $p = 8$; при $p = 4$ (число физических ядер) оно составляет **1,94–2,24**;
- ускорение ниже, чем у OpenMP-версии (этап 3, 2,93 и 2,43): шаги 1 и 3 выполняет один процесс, пока остальные ждут рассылку, а блочную строку приходится упаковывать;
- при $p > 4$ процессы делят физические ядра, поэтому результат слабо растёт и становится шумным. Например, при $n = 1000$, $p = 8$ в этом прогоне $S = 1{,}14$, а в предыдущем прогоне того же кода было $S = 2{,}67$: разложение длится около 0,1–0,2 с при $p = 8$, а MS-MPI ждёт данные активным опросом, отнимая у соседнего процесса время ядра. Замеры сделаны на ноутбуке от батареи;
- объём обменов — не главное ограничение: за разложение при $n = 3000$ рассылается около 36 МБ через общую память при времени разложения в несколько секунд. Ограничения те же, что на этапе 3: общая память и снижение частоты одного ноутбука;
- погрешности `err` и `res` для всех $p$ совпадают с последовательной версией: каждый элемент вычисляется одним процессом в том же порядке вычитаний.

### Как улучшить масштабирование

- запускать на кластере или нескольких виртуальных машинах, где у каждого процесса собственная память и ядро: распределённое хранение здесь уже есть;
- использовать двумерное блочно-циклическое распределение (как в ScaLAPACK): обмены идут только внутри строк и столбцов сетки процессов;
- совмещать обмены с вычислениями (неблокирующий `MPI_Ibcast`), чтобы следующий владелец не ждал окончания шага 4 у остальных.